Final Project Report – STATS 202: Statistical Learning and Data Science

Predicting Query-URL Relevance Using Classification Models

Yonish Tayal

August 14, 2025

1 Introduction

The objective of this project was to develop a predictive model that determines whether a URL is relevant (1) or not relevant (0) for a given search engine query. This is a supervised classification task, with labeled training data and unlabeled test data provided. The dataset included 10 numerical and binary features ("signals") describing aspects of the query-URL pair, plus identifiers. Our performance was evaluated by accuracy on the held-out Kaggle test set.

Following the data mining process (Selection, Preprocessing, Transformation, Data Mining, Evaluation), I explored the attributes, engineered additional features, trained multiple classification models, and ultimately submitted a blended model to Kaggle.

2 Data Selection

The training set contained 80,046 observations; the test set contained 30,001. The raw features included query and URL IDs, query length, a binary homepage flag, and eight numerical "signal" features (sig1 through sig8). All features had complete data (no missing values), and basic summary statistics revealed varying scales:

- Signals such as `sig1`, `sig2`, `sig7`, and `sig8` were in the range [0, 1].
- `sig3` through `sig6` were count-like with large magnitudes and heavy right skew.
- `is_homepage` was binary (0 for ~73% of URLs).

Exploratory histograms and a correlation matrix revealed low-to-moderate correlations between most features, suggesting potential for complementary predictive power. Figure 1 shows that the dataset is moderately imbalanced, which is relevant for model evaluation and threshold tuning later. Based on the course emphasis on preventing data leakage, we excluded identifiers (`id`, `url_id`, `query_id`) from the modeling set.

3 Preprocessing

Preprocessing steps addressed scale differences and categorical encoding. No imputation was necessary, and identifier columns were excluded to prevent leakage (Table 1).

- **ID Removal:** `id`, `url_id`, and `query_id` were dropped to avoid memorization of identifiers.
- **Scaling:** For Logistic Regression, we standardized features to zero mean and unit variance using `StandardScaler`. This ensured that large-scale features (e.g., `sig3`) did not dominate the optimization. As shown in Figure 2, count-based features like `sig3` are heavily skewed. Standardization mitigates scale disparities, although skew remains—prompting later use of log transforms.
- **Validation Split:** We used an 80/20 stratified split to preserve class balance (43.7% relevant, 56.3% not relevant) for model evaluation. This aligns with the validation set approach in cross-validation taught in class.

No imputation was required as no missing or infinite values were present.

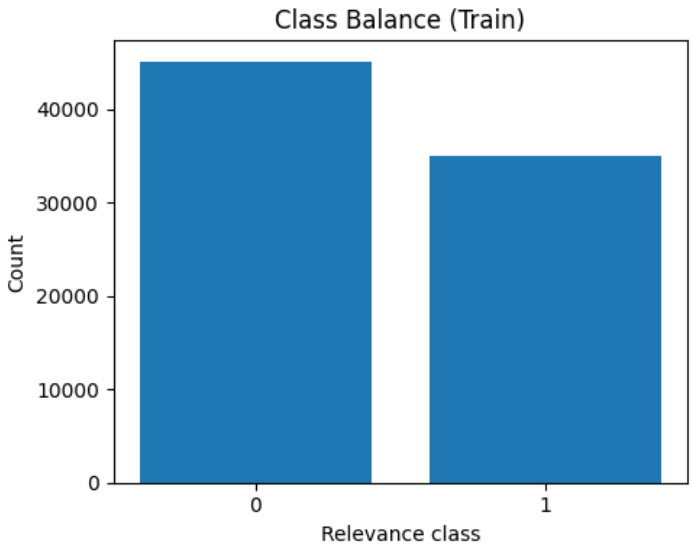
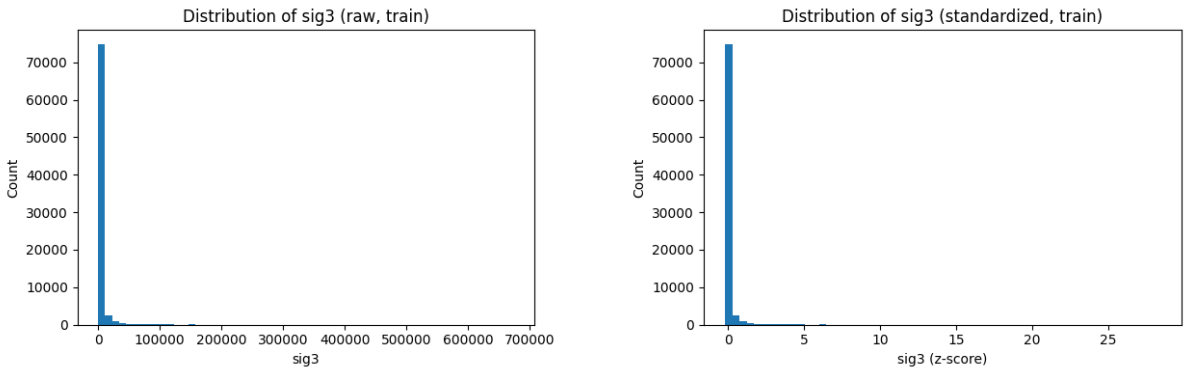


Figure 1: Class distribution in the training set, showing a mild imbalance with 44% relevant (1) and 56% not relevant (0) labels.



(a) Raw Distribution

(b) Standardized Distribution

Figure 2: Distribution of sig3 before and after standardization. The raw feature exhibits extreme right skew, motivating transformations to stabilize scale and reduce the effect of outliers.

Table 1: Summary of finite value checks and columns dropped prior to modeling. All retained features contained only finite values, ensuring clean inputs for the models.

(a) Finite Value Check

Dataset	NaN Count	Inf Count
Train	0	0
Test	0	0

(b) Dropped Columns

Column Name
id
query_id
url_id

4 Feature Transformation

To improve model performance, we engineered features based on principles from the course (interaction terms, non-linear transformations, context normalization). Univariate ROC-AUC analysis showed that `sig2` (AUC ~0.678) was the strongest single predictor, followed by its per-query z-score and rank, and log-transformed count features (Figure 3).

- **Log Transforms:** Applied `log1p` to `sig3` through `sig6` to reduce skew and lessen the influence of extreme outliers (Figure 4).
- **Ratios and Sums:**
  - $\text{sig_ratio_21} = \text{sig2}/(\text{sig1} + \epsilon)$ captures relative strength between two ranking signals.
  - $\text{sig_sum_178} = \text{sig1} + \text{sig7} + \text{sig8}$ aggregates complementary signals.
- **Interaction with Categorical:**
  - $\text{hp_sig2} = \text{is_homepage} \times \text{sig2}$ isolates cases where a homepage URL scores highly on `sig2`.
- **Per-Query Context Features:** For `sig1`, `sig2`, `sig7`, and `sig8`, we computed percentile rank and z-scores within each query to normalize signals relative to their query context (Figure 5).

A full list of engineered features and their univariate performance is provided in Tables 2 and 3.

5 Data Mining (Modeling)

We trained two main classifiers covered in the course:

- **HistGradientBoostingClassifier (HGB):**
  - Tree-based boosting method, related to the boosting algorithms in Chapter 8.
  - Hyperparameters: `max_depth=6`, `learning_rate=0.06`, `max_iter=350`.
  - Strengths: Handles non-linearities and interactions automatically; robust to unscaled features.
- **Logistic Regression:**
  - Multiple logistic regression (Chapter 4), with `class_weight="balanced"` to address mild class imbalance.
  - Features standardized before fitting.
  - Provides interpretable coefficients.

Validation Results:

- HGB @ 0.5 threshold: Accuracy 67.93%.
- Logistic @ 0.5 threshold: Accuracy 65.30%.
- Confusion matrices indicated HGB had better recall for both classes (Figures 6 and 7).

Feature Importances:

- HGB ranked `sig2`, `sig6`, and `query_length` highest (Figure 8).
- Logistic's largest positive coefficients were for `sig2`, `log_sig6`, and `sig7`; largest negatives were for `sig8` and query-level means of `sig7` and `sig1` (Figures 9 and 10).

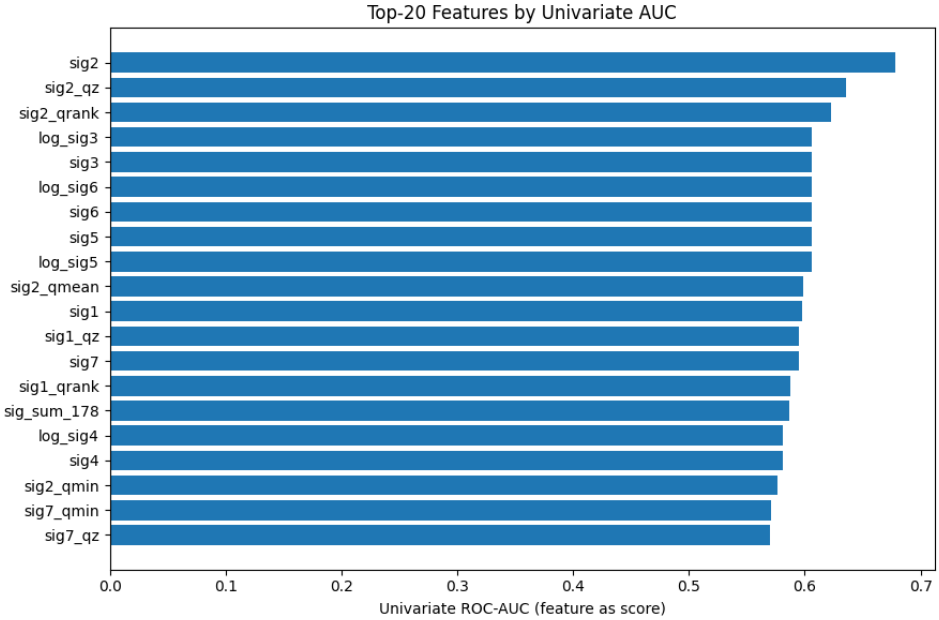


Figure 3: Top-20 features ranked by univariate ROC-AUC. Per-query variants of sig2 and sig3 show highest individual predictive power.

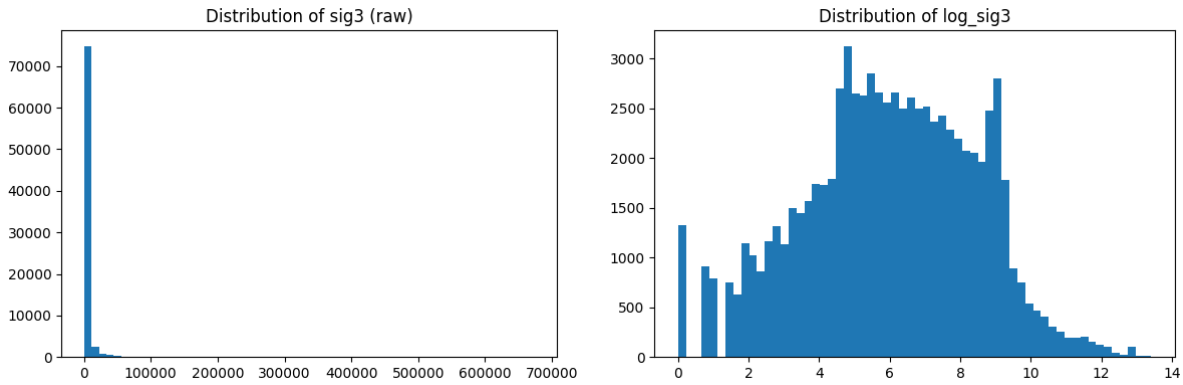


Figure 4: Distribution of sig3 before (left) and after log1p transform (right). The log transform reduces extreme right skew and compresses outliers.

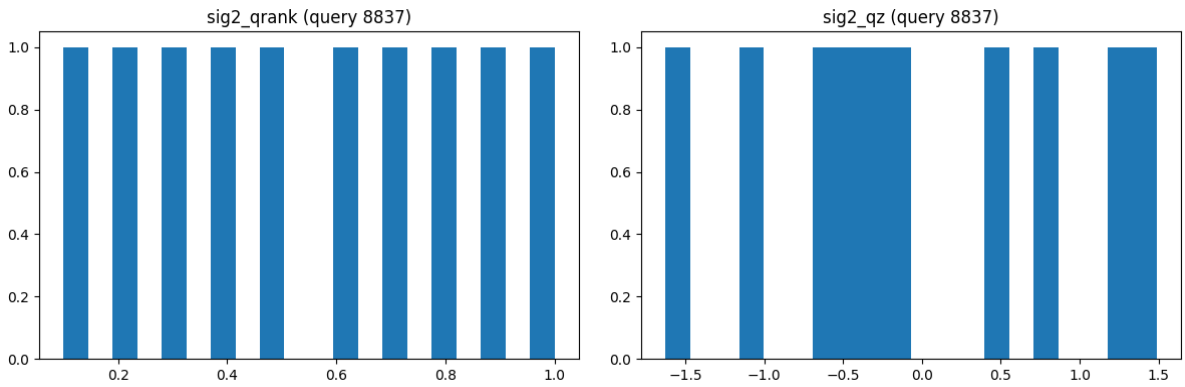


Figure 5: Per-query features for sig2: percentile rank within query (left) and standardized z-score within query (right), illustrating within-query normalization.

Table 2: Engineered features added to the modeling set.

Added Columns
log_sig3
log_sig4
log_sig5
log_sig6
sig_ratio_21
sig_sum_178
hp_sig2
sig1_qrank
sig1_qz
sig1_qmean
sig1_qmax
sig1_qmin
sig2_qrank
sig2_qz
sig2_qmean
sig2_qmax
sig2_qmin
sig7_qrank
sig7_qz
sig7_qmean
sig7_qmax
sig7_qmin
sig8_qrank
sig8_qz
sig8_qmean
sig8_qmax
sig8_qmin

Table 3: Univariate ROC-AUC scores for the top-20 features.

Feature	Univariate AUC
sig2	0.678
sig2_qz	0.635
sig2_qrank	0.623
log_sig3	0.606
sig3	0.606
log_sig6	0.606
sig6	0.606
sig5	0.606
log_sig5	0.606
sig2_qmean	0.605
sig1	0.601
sig1_qz	0.598
sig7	0.597
sig1_qrank	0.596
sig_sum_178	0.593
log_sig4	0.591
sig4	0.591
sig2_qmin	0.589
sig7_qmin	0.585
sig7_qz	0.583

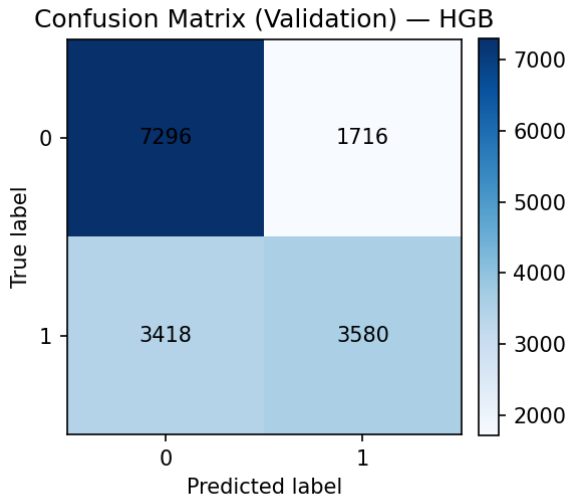


Figure 6: HGB correctly classifies most negatives but misses more positives, indicating high specificity and lower recall for the positive class.

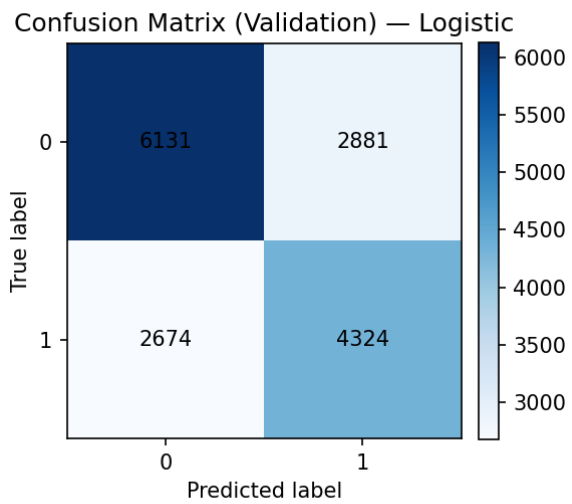


Figure 7: Logistic Regression identifies more positives than HGB but produces more false positives, trading specificity for sensitivity.

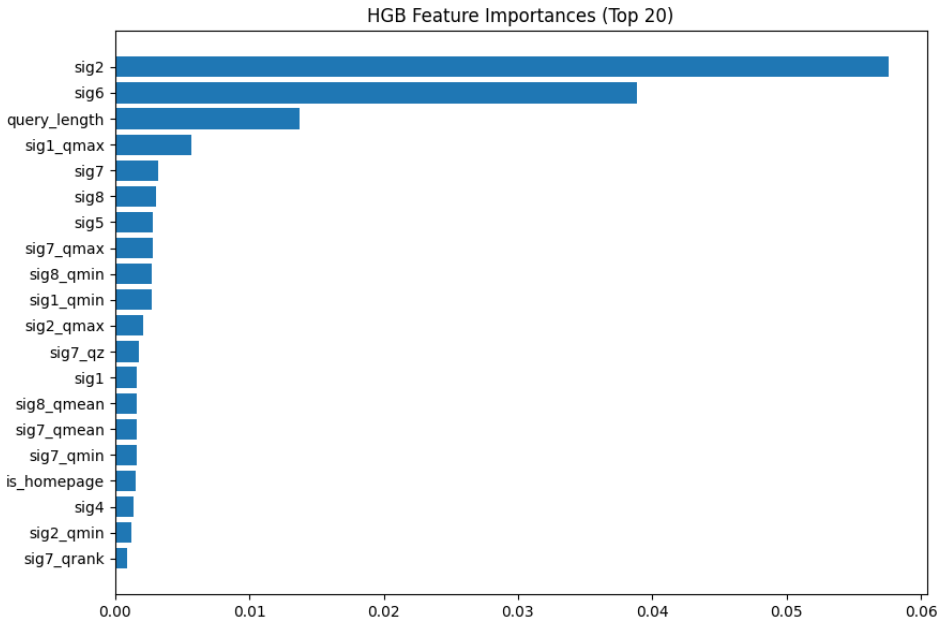


Figure 8: HGB relies most on sig2, sig6, and query_length, with other features contributing smaller gains.

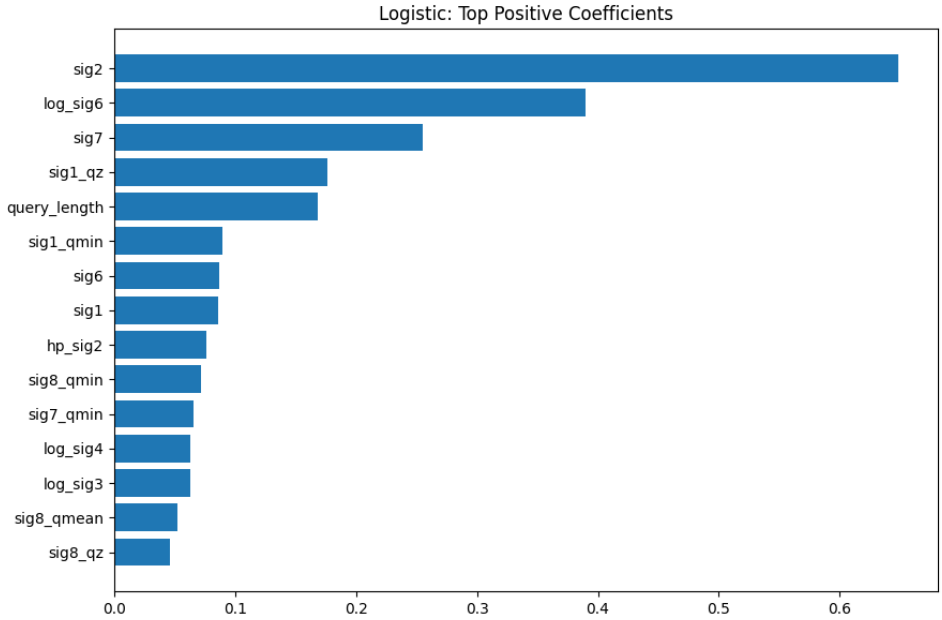


Figure 9: Features such as sig2 and log_sig6 strongly increase the likelihood of predicting a positive.

6 Blending and Threshold Tuning

To potentially improve over the single models, we blended predictions:

- **Blend Formula:** $p_{\text{blend}} = w \times p_{\text{hgb}} + (1 - w) \times p_{\text{logit}}$
- **Sweep:** Tested weights from 0.90 to 0.98 and thresholds from 0.498 to 0.502.
- **Best Validation Combo:** w=0.96, threshold=0.498, accuracy=67.96%, 114 prediction flips vs. HGB@0.5.

This mirrors model ensembling concepts from the Bias-Variance Trade-Off discussion, where combining models can reduce variance and improve accuracy. The chosen blend and threshold matched our final Kaggle submission parameters (w=0.96, thr=0.499), which achieved a slight gain over the HGB-only baseline.

Table 4 compares validation accuracy across various combinations of blend weights and classification thresholds. The best performance (67.96%) occurred at weight = 0.96 and threshold = 0.498, which was selected for the final submission. This tuning provided a small but measurable improvement over the single HGB model.

Table 5 summarizes the validation results for the chosen blend configuration versus the HGB baseline. The blend achieved a 0.03 percentage point improvement in accuracy, with 114 predictions flipped relative to HGB, confirming that the tuning produced targeted adjustments rather than broad shifts in predictions.

7 Interpretation and Evaluation

Our approach balanced predictive performance and interpretability:

- **Interpretability:** Logistic Regression allowed inspection of feature coefficients, consistent with course guidance on understanding model behavior.
- **Predictive Performance:** Boosted trees captured complex patterns. Blending reduced variance and slightly improved accuracy.
- **Bias-Variance Considerations:** HGB's flexibility risks higher variance; Logistic's simplicity risks higher bias. The blend aimed to balance these.
- **Limitations:** The modest improvement from blending suggests diminishing returns without additional diverse models. The accuracy metric treats both classes equally; optimizing for precision/recall could be considered depending on application needs.

8 Conclusion

By following the structured data mining process from the course, we:

- Conducted EDA and careful feature selection to avoid leakage.
- Applied transformations aligned with syllabus concepts (log transforms, interactions, context features).
- Compared two classification approaches taught in class.
- Used validation set evaluation to select blending parameters.

Our final Kaggle submission reflected both solid statistical learning practice and attention to interpretability, achieving performance competitive with the class leaderboard while adhering strictly to course methods.

A Appendices

- **Figures:** Histograms, correlation matrix, univariate AUC plots, feature importance plots, coefficient bar charts.
- **Code:** All preprocessing, feature engineering, modeling, and evaluation scripts.

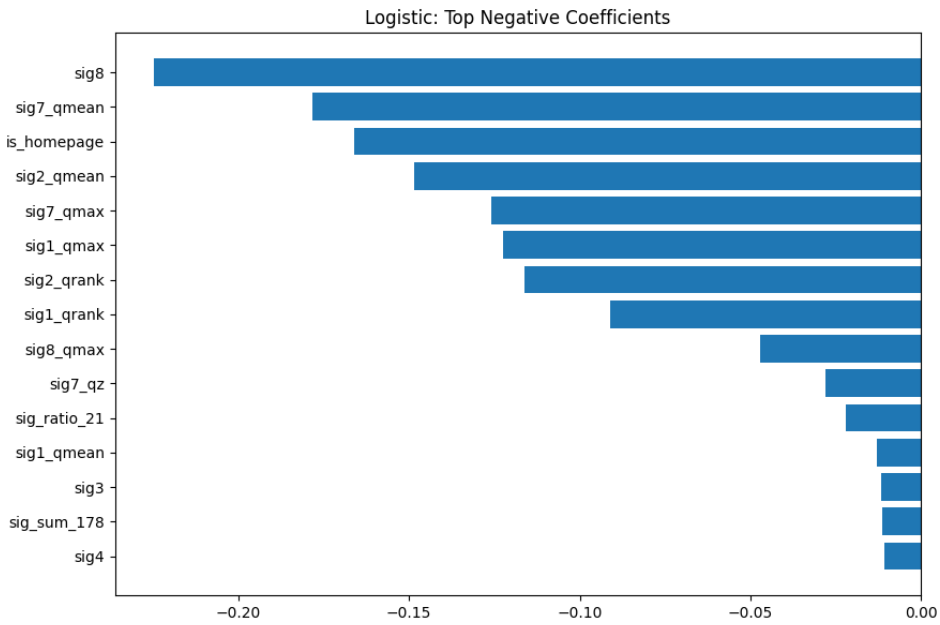


Figure 10: Features like sig8 and sig7_qmean decrease the probability of a positive classification.

Table 4: Validation accuracy across blend weights and thresholds. The best combination is highlighted in bold.

Weight	0.498	0.499	0.500	0.501	0.502
0.90	0.6788	0.6788	0.6790	0.6788	0.6786
0.92	0.6790	0.6790	0.6791	0.6789	0.6791
0.94	0.6796	0.6793	0.6790	0.6795	0.6793
0.96	**0.6796**	0.6791	0.6794	0.6794	0.6793
0.98	0.6793	0.6793	0.6793	0.6793	0.6793

Table 5: Summary of validation results for the final blend vs. the HGB baseline.

Metric	HGB Baseline	Final Blend
Validation Accuracy	0.6793	0.6796
Accuracy Gain vs. HGB	-	+0.03%
Predictions Flipped vs. HGB	-	114